Zoom Transcript Pipeline

Automated delivery of Zoom Phone call transcripts, meeting transcripts and Team Chat records into SharePoint

Sensible Care Pty Ltd

1. What this system does

Zoom records and transcribes calls and meetings, and holds every Team Chat conversation, but keeps all of it inside Zoom, where it is awkward to search, impossible to retain under policy, and invisible to the rest of Microsoft 365. This pipeline copies all three into a SharePoint document library in Sensible Care's own tenant, filed by the team the record belongs to and the week it happened.

Three kinds of record are collected: **Zoom Phone** call transcripts, **Zoom meeting** transcripts, and **Zoom Team Chat** conversations for a list of work-related channels chosen by the division managers. Personal and ad-hoc group chats are never collected.

Three things follow from that. Transcripts become searchable alongside everything else in SharePoint; Microsoft Purview retention labels can be applied to them like any other document; and no third party ever holds the data, because it moves directly from Zoom to Sensible Care's tenant.

Property	Value
Source	Zoom Phone transcripts, meeting transcripts and Team Chat, via the Zoom API
Destination	SharePoint library *Call Transcripts*, in three top-level folders
Filing	`{Team}/{ISO year}-W{week}/` — team, then week, for all three
Time zone	Australia/Melbourne — weeks are local, not UTC
Format	Plain text (`.txt`) with a metadata header
Volume	Roughly 640 call transcripts per day; meetings and chat are far smaller
History held	Calls from July 2025, meetings from May 2025, chat from July 2025. Zoom holds nothing earlier
Hosting	Azure Container Apps, region `australiaeast`

2. The moving parts

Call transcripts arrive by two routes — a webhook for speed and a nightly job as the safety net — sharing one piece of delivery logic, so there is only ever one answer to “which folder, and has this already been delivered?”. Meeting transcripts use the same two routes. Team Chat is collected by the nightly job only, since a week’s conversation is written as a single file.

The webhook — real time

An always-on web service. Zoom calls it the moment a transcript finishes processing, and the transcript is usually in SharePoint within seconds. This is the primary path; almost everything arrives this way.

The nightly job — the safety net

Runs once a day and re-checks the last two days. Anything the webhook missed — because Zoom's delivery failed, or

the service was restarting — is picked up here. It then seals the previous week's manifests and reconciles what Zoom holds against what SharePoint holds.

History

Everything recorded before the pipeline existed was loaded separately, as a one-off. That work is complete: all available call transcripts, 277 meeting transcripts and 1,394 weekly chat records have been delivered. Zoom holds nothing older, so there is no further history to collect.

3. How a transcript travels

1. **Zoom signals.** A `phone.recording_transcript_completed` event is posted to the webhook, or a scheduled job queries `GET /v2/phone/recordings` for a date range.
2. **The sender is verified.** Webhook requests carry an HMAC-SHA256 signature over `v0:{timestamp}:{body}`. A bad signature, or a timestamp too far from now, is rejected as a possible replay.
3. **The owner is resolved to a team.** The recording names a Zoom user or a call queue. That is turned into an email address, and the email is looked up in the team map.
4. **Prior delivery is checked.** The destination week is listed in SharePoint. If this recording's id is already there, it is skipped.
5. **The transcript is fetched and rendered.** Zoom returns JSON; the pipeline converts it to readable text and prepends a metadata header.
6. **It is uploaded** to `{Team}/{ISO year}-W{week}/`, creating the folder if needed.
7. **A manifest row is recorded** for the weekly summary.

> **Transcript text is never written to local disk.** It is streamed from Zoom, rendered in memory, and sent to SharePoint. Nothing remains on the machine running the pipeline.

4. Where files land

Each kind of record has its own top-level folder in the same library, and inside it the same two levels: the team, then the week.

```
Call Transcripts/

  Care Partner Team/

    2026-W39/

      143301_inbound_+61400000000_to_200_d8d963272aa1.txt

      _manifest_2026-W39_Care Partner Team.csv

  Rostering/

    2026-W39/

  _Unassigned/

    2026-W39/
```

```
  _config/
      teams.csv
      chat_channels.csv

Meeting Transcripts/
  Care Partner Team/
    Meeting - 2026 - W41/
      092316_Care Management Workshop_care_122c0b64.txt
      _manifest_2026-W41_Care Partner Team.csv

Chat Transcripts/
  Accounts Team/
    Chat - 2026 - W41/
      CMs + ACCOUNTS_85e2d6be48fb.txt
      _manifest_2026-W41_Accounts Team.csv
```

Call transcripts are named `HHMMSS_direction_caller_to_callee_shortid.txt` and meeting transcripts `HHMMSS_topic_host_shortid.txt`, both in Melbourne local time. Leading with the time makes a folder sort chronologically, and the twelve-character id on the end makes collisions impossible.

Chat is one file per channel per week, named for the channel, holding that week's whole conversation. Replies are shown indented beneath the message they answer, so threads stay readable.

> **Channels shared between teams.** Where a channel belongs to two divisions, the record is filed under *both* — the same file appears in each team's folder. Every chat file names the teams it belongs to at the top, so it is clear this is intended and not a stray duplicate.

> **Why ISO weeks and not months.** The folder name pairs the ISO year with the ISO week, never the calendar year. 1 January 2027 falls in ISO week 53 of 2026; writing `2027-W53` would create a folder that should not exist. The code enforces the pairing.

5. Weekly manifests

Each team gets one CSV per week for each kind of record, listing everything delivered. For calls: recording id, time, direction, duration, team, filename and upload time. For meetings: the topic, host and duration. For chat: the channel, how many messages and threads it held, and the period it covers.

A manifest is written only once the week has closed and a grace period has passed, so that late arrivals are included. Once written it is never rewritten. Anything that arrives afterwards goes into a numbered supplement file beside it, containing only the genuinely new rows.

6. Azure infrastructure

Resource	Name	Purpose
Resource group	rg-zoom-transcripts	Holds everything
Container Apps environment	cae-zoom-transcripts	Shared network and logging
Container app	app-transcripts-webhook	Always-on webhook receiver
Job (scheduled)	job-transcripts-nightly	Daily safety net, seal, reconcile, meetings, chat
Job (manual)	job-transcripts-manual	Ad-hoc operator runs and one-off history
Container registry	Azure Container Registry	Stores the built image
File share	Azure Files, mounted at `/data`	State that must survive restarts
Logs	Log Analytics	Console output from every run

Containers are destroyed when they finish. Everything that must outlive a run is written to the Azure Files share, which persists independently of them.

The nightly job runs its stages in order — calls, manifests, reconciliation, meetings, then chat. Chat is last because it is the longest and least urgent stage, so if a run is ever cut short it is chat that waits for the next night rather than the call transcripts.

7. Schedules

Azure runs cron in **UTC and does not observe daylight saving**. The UTC column is what is configured; the Melbourne column is the same instant expressed locally, and shifts when Melbourne changes clocks.

Job	UTC	Melbourne
Nightly	16:00 daily	02:00 next day (03:00 in daylight time)
Webhook	Always running; no schedule	